Convolutional Neural Network

S. R. Mahadeva Prasanna

Professor, Dept. of EE, IIT Dharwad

prasanna@iitdh.ac.in

September 16, 2024

Some Good Resources for NNDL

- Mithesh M. Khapra, "Deep Learning Part I and II" NPTEL Course.
- Ali Ghodsi, "Deep Learning", University of Waterloo Course.
- Duda and Hart, "Pattern Classification" Wiley, 2001.
- C. M. Bishop, "Neural Networks for Pattern Recognition", 1995.
- B. Yegnanarayana, "Artificial Neural Networks", PHI, 1999.
- C. M. Bishop, "PRML", Springer, 2006.
- Ian Goodfellow, et. al. , "Deep Learning", MIT Press, 2016.
- C. M. Bishop & H. Bishop, "Deep Learning", Springer, 2024
- Simon J. D. Prince, "Understanding Deep Learning", MIT Press, 2024

Introduction to Convolutional Neural Networks

- Convolutional Neural Networks (CNNs) are for processing 2D data like image. [1].
- CNNs use convolution operation in place of matrix multiplication in at least one of their layers.
- The output of convolution of input with the kernel is feature map.
- The goal is to learn appropriate values for the kernel using a suitable learning algorithm.
- Each kernel will represent a meaningful pattern once the learning is complete.

Why the Name Convolution Network?

- Generally, matrix multiplication across pair of layers.
- CNN employs convolution as alternative to matrix multiplication.
- Convolution layer have three stages: Convolution, Detection and Pooling.
- Convolution is a mathematical operation used in signal processing.
- CNNs suitable for processing grid (block) like input topology.
- 1D time-series data mapped to 2D, 2D image data, 3D video data.

Engineering Motivation for CNNs

- Can I modify structure of neural networks to mimic deep learning strategy of human brain ?
- Learning is hierarchy of concepts.
- First learn simple concepts from input data.
- Then Use these simple concepts to create abstract concept.
- Significantly reduce no of parameters required to be computed as required in traditional neural network (dense)
- Accordingly, reduce overfitting problem in case of less amount of data

PS CNNs were first introduced in 1980s by Yann LeCun. LeCun had built on the work done by Kunihiko Fukushima, a Japanese scientist who, a few years earlier, had invented the neocognitron, a very basic image recognition neural network.

Motivation for CNNs: Neuroscientific Basis

- Primary visual cortex (PVC) part of the brain responsible for visual scene processing.
- Performs significantly advanced processing of visual input.
- Images are formed by light arriving in the eye and stimulating the retina, the light-sensitive tissue in the back of the eye.
- Neurons in the retina perform some simple preprocessing of the image but do not substantially alter the way it is represented.
- Image then passes through optic nerve and reaches primary visual cortex.

Motivation for CNNs: Neuroscientific Basis

- PVC is arranged in a spatial map and has a two-dimensional structure mirroring structure of image in the retina.
- CNNs capture this property by having their features defined in terms of two dimensional maps.
- PVC contains many simple cells performing a linear function of the image in a small, spatially localized receptive field.
- Detector units of a CNN are designed to emulate these properties.
- PVC also contains many complex cells that respond to features detected by simple cells, but are invariant to small shifts in the position of the feature.
- This inspires the pooling units of CNNs.

Motivation for CNNs: Neuroscientific Basis

- Generally believed that the same basic principles of PVC apply to other areas of the visual system.
- Basic strategy of detection followed by pooling is repeatedly applied as we move deeper into the brain.
- As we pass through multiple anatomical layers of the brain, eventually find cells that respond to some specific concept and are invariant to many transformations of the input.

Input and Kernel

- In convolutional network terminology, the first argument $x(n, m)$ to the convolution is often referred to as the input.
- The second argument $h(n, m)$ is termed as the kernel.
- Input is a multidimensional array of data and kernel is a multidimensional array of parameters.
- These multidimensional arrays are termed as tensors.
- These functions ($x(n, m)$ and $h(n, m)$) are assumed to be zero everywhere except for the grid of finite set of points.
- The goal of deep learning is to learn the appropriate values for the kernel using a suitable learning algorithm.

Sparseness due to Convolution for Neural Networks

- Traditional neural network layers use matrix multiplication by a matrix of parameters.
- Convolutional networks, however, typically have sparse interactions. This is accomplished by making the kernel smaller than the input.
- Need to store fewer parameters, which both reduces the memory requirements of the model and improves its statistical efficiency.
- Input m, output n, kernel k. Traditional NN, $m \times n$ and $k \times n$, where later results in far few parameters and significantly less computations.
- In a deep convolutional network, units in the deeper layers may indirectly interact with a larger portion of the input.

Important Points Supporting Convolutional Neural Networks

- Spatial Knowledge : Class specific information is in spatial domain.
- Sparse Interaction: Information in spatial domain is only among neighbourhood of pixel values or local region.
- Need to Store Fewer Parameters : Sharing of parameters across different neurons in a given layer.

Parameter Sharing due to Convolution for Neural Networks

- Parameter sharing refers to using the same parameter for more than one function in a model.
- In a traditional neural net, each element of the weight matrix is used exactly once when computing the output of a layer.
- In a convolutional neural net, each member of the kernel is used at every position of the input.
- The parameter sharing means rather than learning a separate set of parameters for every location, learn only one set.

Pooling in CNNs

- A typical layer of a convolutional network consists of three stages.
- In the first stage, the layer performs several convolutions in parallel to produce a set of linear activations.
- In the second stage, each linear activation is run through a nonlinear activation function, such as the ReLU activation function.
- This stage is sometimes called the detector stage.
- In the third stage, we use a pooling function to modify the output of the layer further.
- A pooling function replaces the output of the net at a certain location with a summary statistic of the nearby outputs.

Pooling in CNNs

- For example, the max pooling operation reports the maximum output within a rectangular neighborhood.
- Other popular pooling functions: Avg. of a rectangular neighborhood, L^2 norm of a rectangular neighborhood, or a weighted average based on the distance.
- Pooling helps to make the representation become approximately invariant to small translations of the input.
- Invariance to translation $\implies$ if input is translated by a small amount, values of most of pooled outputs do not change.
- Invariance to local translation a very useful property if we care more about whether some feature is present than exactly where it is.

Pooling in CNNs

- Because pooling summarizes responses over a whole neighborhood, it is possible to use fewer pooling units than detector units.
- This improves the computational efficiency of the network because the next layer has roughly k times fewer inputs to process.
- For many tasks, pooling is essential for handling inputs of varying size.
- For example, if we want to classify images of variable size, the input to the classification layer must have a fixed size.
- This is accomplished by varying size of an offset between pooling regions so that classification layer always receives same number of summary statistics regardless of the input size.

Training CNNs

- Most expensive part of CNN training is learning the features.
- Supervised training with gradient descent needs every gradient step a complete run of forward and backward propagations through entire network.
- One way to reduce the cost of CNN training is to use features that are not trained in a supervised fashion.

Information About Kernels

- Three basic strategies for obtaining convolution kernels without supervised training.
- One is to simply initialize them randomly.
- Another is to design them by hand, for example by setting each kernel to detect edges at a certain orientation or scale.
- Finally, one can learn the kernels with an unsupervised criterion.
- Unsupervised Ex.: Apply k-means clustering to small image patches, then use each learned centroid as a convolution kernel.
- Learning the last layer is then typically a convex optimization problem, assuming the last layer is something like logistic regression or an SVM.

Training using Random Kernels

- Random kernels often work surprisingly well in convolutional networks.
- Layers consisting of convolution followed by pooling naturally become frequency selective and translation invariant when assigned random weights.
- They argue that this provides an inexpensive way to choose the architecture of a convolutional network.
- First evaluate the performance of several CNN architectures by training only the last layer.
- Then take the best of these architectures and train the entire architecture using a more expensive approach.

Greedy Layer-wise Training

- An intermediate approach is to learn the features, but using methods that do not require full forward and back-propagation at every gradient step.
- As with multilayer perceptrons, use greedy layer-wise pretraining, to train the first layer in isolation.
- Then extract all features from the first layer only once, then train the second layer in isolation given those features, and so on.

Block Diagram of CNN

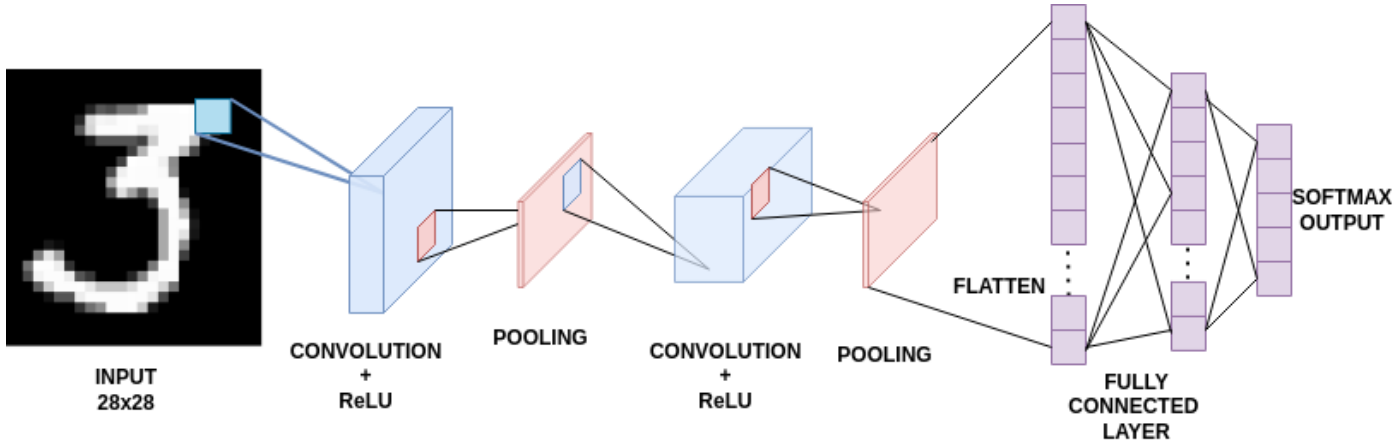


Figure: Block Diagram of CNN with 2 convolution layers and fully connected layer

- Each layer of CNN consists of three stages.
  - Convolution to produce activation values.
  - Non-linear output function like ReLU applied on activation values.
  - Pooling function for data reduction and making it translation invariant.

Motivation for CNNs

- Knowledge: class specific information is in spatial domain.
- Sparse Interaction: Information in spatial domain is only among the neighbourhood of pixel values or local region.
- Parameter Sharing: Sharing of parameters across different neurons in a given layer. The learned parameters are reused rather than learning new parameters.

Convolution Operation on Images

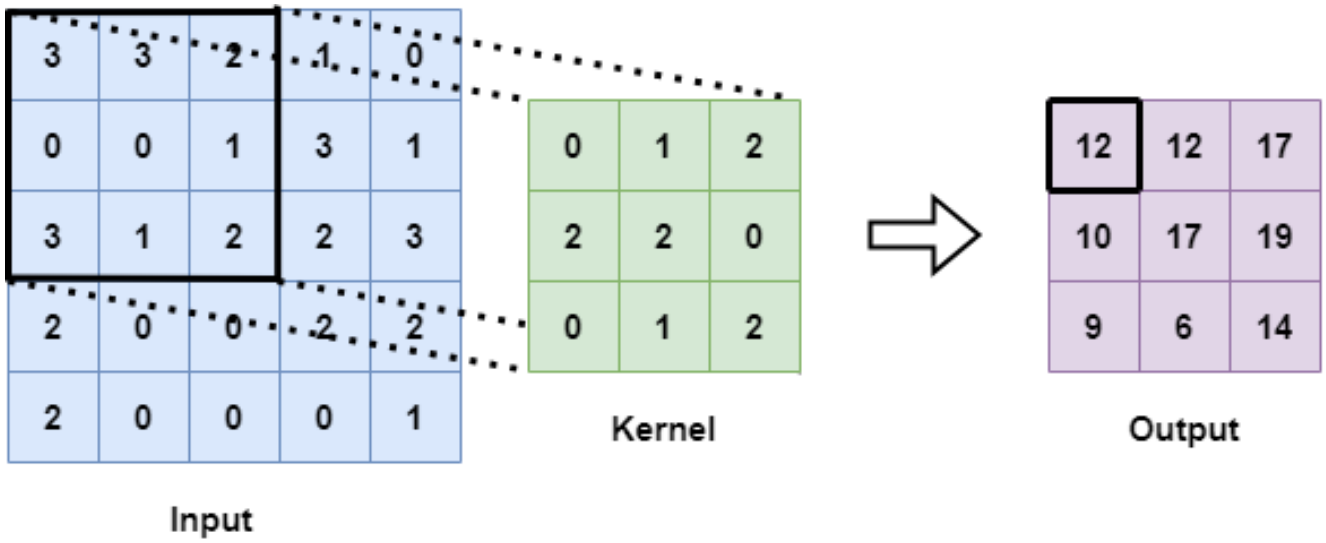


Figure: Convolution operation on image

- Formula for convolution operation [1].

$$S(i,j) = K * I(i,j) = \sum_m \sum_n I(i+m, j+n) K(m,n)$$

- I is input, K is kernel and S is output.

.

Pooling Operation on Images

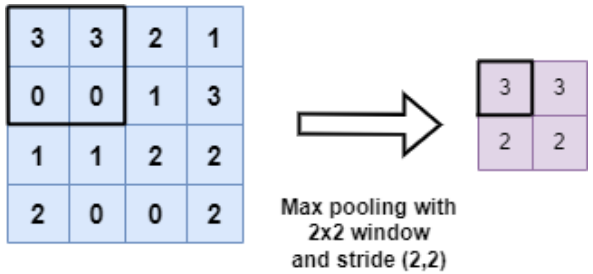


Figure: Pooling operation on image

- Pooling replaces output with a summary statistic of nearby outputs.
- Max pooling replaces with max output.
- Other pooling functions: Average of neighbourhood, L2 norm of neighbourhood, or a weighted average based on distance.
- Pooling makes representation invariant to small translations in input. CNN on translated images yields better performance than FFNN.

Feature map sizes

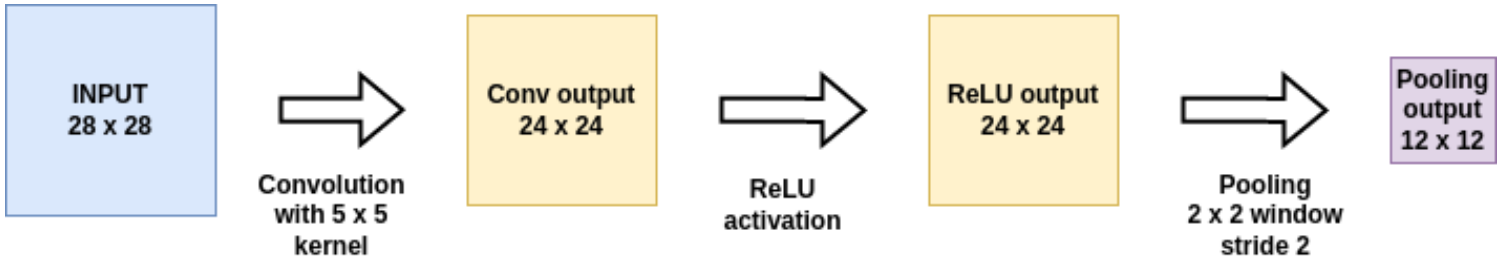


Figure: Size of convolution layer output

- Size of image reduces after convolution and pooling operation. Not desirable since it loses information along borders of image.
- Size of image can be kept constant using zero/same padding and by pooling with stride 1.
- Size of output of convolution layer [2]. $O = \frac{I-K+2P}{S} + 1$
- O, I, K, P and S are size of output, input, kernel, number of padded layers, and stride, respectively.

Comparison of CNN with FFNN

- MNIST handwritten digit classification task is used for comparison.

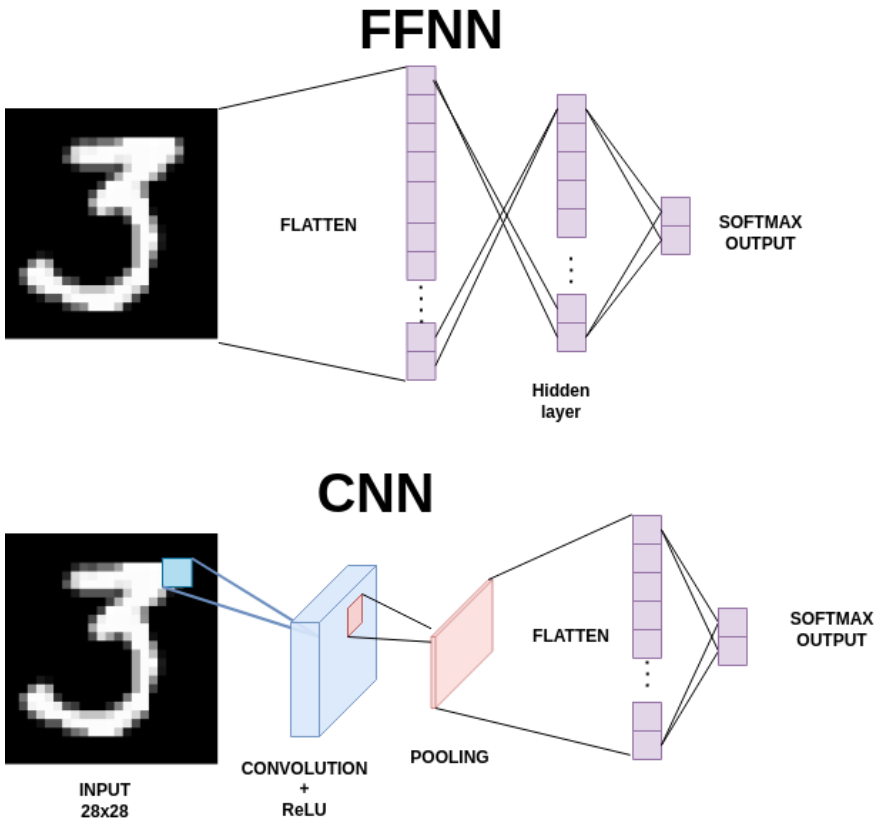


Figure: Comparison of FFNN and CNN with one hidden layer

Results of CNN and FFNN: Two classes

- Data of two classes are taken from MNIST dataset.
- Both models have one hidden layer and are trained up to 10 epochs with same learning rate.
- In FFNN, 28x28 image is flattened into a vector and passed to hidden layer.
- Performance of CNN is better than FFNN with fewer parameters. FFNN loses spatial information when the image is flattened.

Table: Comparison of FFNN and CNN for two classes

Parameter	**FFNN**	**CNN**
Number of parameters	39352	3142
Performance on test set	96.47%	98%
Performance on translated test set	80.42%	88.26%

Learned distribution: Hidden layer output

- Output of hidden layers of both FFNN and CNN are compared using tSNE plot.
- CNN shows better separation.

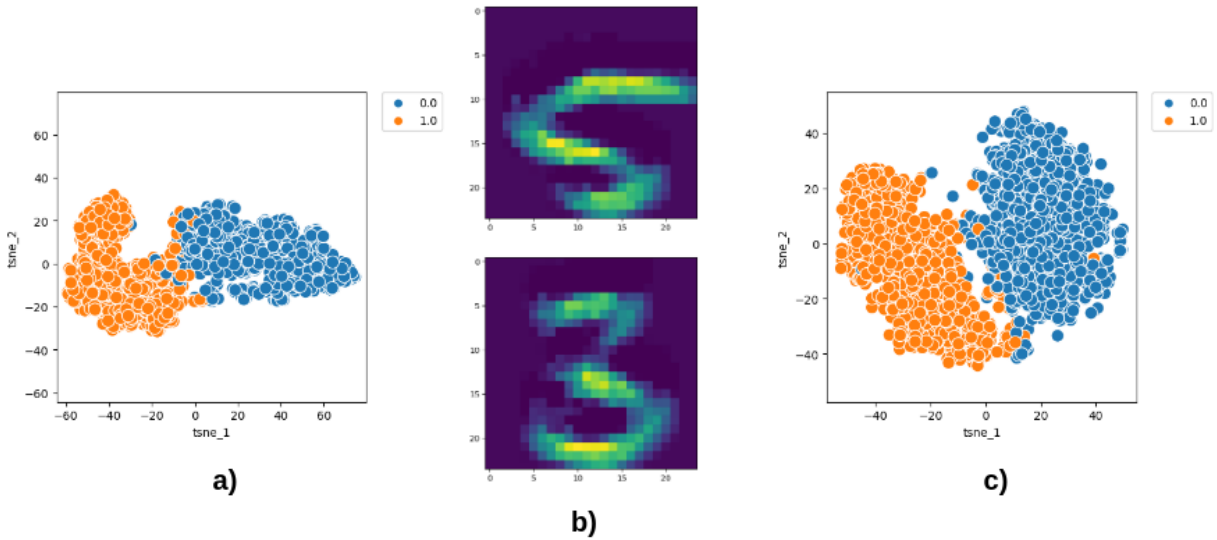


Figure: a) T-SNE plot of output of FFNN hidden layer b) Output of Convolution layer c) T-SNE plot of flattened output of pooling layer

Results of CNN and FFNN: Ten classes

- The results obtained for the entire MNIST dataset with ten classes.

Table: Comparison of FFNN and CNN with one hidden layer for ten classes

Parameter	**FFNN**	**CNN**
Number of parameters	39760	14670
Performance on test set	96.76%	98.43%
Performance on translated test set	63.91%	77.05%

Comparison with 2 hidden layers

- An additional hidden layer is added to FFNN and another convolution layer is added to CNN.
- Entire MNIST dataset is taken for classification.
- Performance has increased with addition of another hidden layer.
- Performance of FFNN degraded for translated test set, and increased for CNN.

Table: Comparison of FFNN and CNN with two hidden layers for ten classes

Parameter	**FFNN**	**CNN**
Number of parameters	404760	21840
Performance on test set	97.61%	98.81%
Performance on translated test set	62.5%	88.83%

Alexnet : ImageNet Classification with Deep CNN

- Large & deep CNN to classify 1.2 million high-resolution images in ImageNet LSVRC-2010 contest into 1000 different classes.
- On test data, top-1 and top-5 error rates of 37.5% and 17.0%, better than state-of-the-art.
- NN has 60 million parameters, five convolutional layers, some by max-pooling layers, and three fully-connected layers with a final 1000-way softmax.
- A variant in ILSVRC-2012 achieved a winning top-5 test error rate of 15.3%, compared to 26.2% achieved by second-best entry.

Alex Krizhevsky, et. al, "ImageNet Classification with Deep Convolutional Neural Networks" Proc. NIPS 2012

Why CNN ?

- To learn about thousands of objects from millions of images, need a model with a large learning capacity.
- CNNs capacity can be controlled by varying their depth and breadth
- Compared to standard FFNN with similarly-sized layers, CNNs have much fewer connections and parameters and easier to train
- Despite attractive qualities of CNNs have still been prohibitively expensive to apply in large scale to high-resolution images.
- Luckily, GPUs paired with large dataset like ImageNet facilitate training of interestingly-large CNNs without severe overfitting.

Alex Krizhevsky, et. al, "ImageNet Classification with Deep Convolutional Neural Networks" Proc. NIPS 2012

Alexnet Architecture

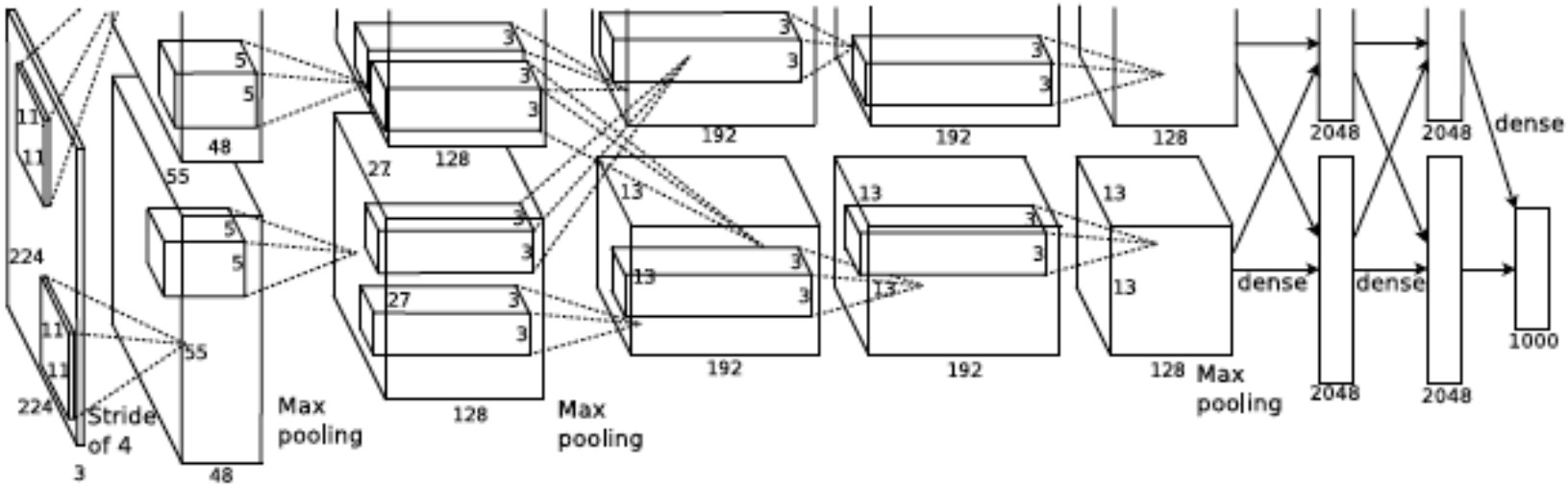


Figure: Alexnet Architecture

- Net contains eight layers; first five are convolutional and remaining three are fully connected.
- Output of last fully-connected layer is fed to a 1000-way softmax.

Alex Krizhevsky, et. al, "ImageNet Classification with Deep Convolutional Neural Networks" Proc. NIPS 2012

Alexnet Architecture

- First convolutional layer filters the $224 \times 224 \times 3$ input image with 96 kernels of size $11 \times 11 \times 3$ with a stride of 4 pixels.
- Second convolutional layer filters with 256 kernels of size $5 \times 5 \times 48$.
- Third convolutional layer has 384 kernels of size $3 \times 3 \times 256$.
- Fourth convolutional layer has 384 kernels of size $3 \times 3 \times 192$
- Fifth convolutional layer has 256 kernels of size $3 \times 3 \times 192$.
- Third, fourth, and fifth convolutional layers are connected to one another without any intervening pooling or normalization layers.
- The fully-connected layers have 4096 neurons each.

Alex Krizhevsky, et. al, "ImageNet Classification with Deep Convolutional Neural Networks" Proc. NIPS 2012

Alexnet Performance on ILSVRC-2010

Model	Top-1	Top-5
Sparse coding [2]	*47.1%*	*28.2%*
SIFT + FVs [24]	*45.7%*	*25.7%*
CNN	**37.5%**	**17.0%**

Figure: Alexnet perfomance on ILSVRC-2010 Challenge Dataset

- Alexnet achieves top-1 and top-5 test set error rates of 37.5% and 17.0%.
- 47.1% and 28.2% with an approach that averages predictions produced from six sparse-coding models trained on different features
- 45.7% and 25.7% with an approach that averages the predictions of two classifiers trained on Fisher Vectors (FVs)

Alex Krizhevsky, et. al, "ImageNet Classification with Deep Convolutional Neural Networks" Proc. NIPS 2012

Alexnet Performance on ILSVRC-2012

Model	Top-1 (val)	Top-5 (val)	Top-5 (test)
SIFT + FVs [7]	—	—	*26.2%*
1 CNN	40.7%	18.2%	—
5 CNNs	38.1%	16.4%	**16.4%**
1 CNN*	39.0%	16.6%	—
7 CNNs*	36.7%	15.4%	**15.3%**

Figure: Alexnet perfomance on ILSVRC-2012 Challenge Dataset

- Alexnet achieves a top-5 error rate of 18.2%.
- Averaging predictions of five similar CNNs gives 16.4%.
- Training one CNN, with an extra sixth convolutional layer gives an error rate of 16.6%.
- Averaging predictions of two CNNs pre-trained on entire Fall 2011 release with aforementioned five CNNs gives an error rate of 15.3%.
- Second-best contest entry achieved 26.2%, averages predictions of several classifiers.

Alex Krizhevsky, et. al, "ImageNet Classification with Deep Convolutional Neural Networks" Proc. NIPS 2012

VGGNET: Deep CNNs for Large Scale Image Recognition

- Investigate effect of CNN depth on its accuracy in large-scale image recognition setting.
- A thorough evaluation of networks of increasing depth using an architecture with very small (3×3) convolution filters.
- A significant improvement on prior-art configurations can be achieved by pushing depth to 16–19 weight layers.
- Top performance in ImageNet Challenge 2014.
- Representations generalise well to other datasets, where they achieve state-of-the-art results.

Karen Simonyan, et al, "Very Deep CNNs for Large Scale Image Recognition", ICLR 2015

VGGNET vs Alexnet

- No. of attempts to improve Alexnet
- ILSVRC-2013 (Zeiler, 2013 ; Sermanet, 2014) utilised smaller receptive window size and smaller stride of first convolutional layer.
- Training and testing networks densely over the whole image and over multiple scales (Sermanet, 2014; Howard, 2014).
- VGGNet address another important aspect of ConvNet architecture design – its depth.
- Fix other parameters of architecture, and steadily increase depth of network by adding more convolutional layers.
- Feasible due to very small (3×3) convolution filters in all layers.

Karen Simonyan, et al, "Very Deep CNNs for Large Scale Image Recognition", ICLR 2015

VGGNET Architecture

- Input to ConvNets is a fixed-size 224×224 RGB image.
- Use filters with a very small receptive field: 3×3.
- The convolution stride is fixed to 1 pixel.
- Spatial padding of conv. layer input is such that the spatial resolution is preserved after convolution.
- Spatial pooling is carried out by five max-pooling layers. Max-pooling is performed over a 2×2 pixel window, with stride 2.
- CNN layers followed by 3 Fully-Connected (FC) layers: first two have 4096 channels each, third performs 1000-way ILSVRC classification.
- Final layer is the soft-max layer.

Karen Simonyan, et al, "Very Deep CNNs for Large Scale Image Recognition", ICLR 2015

Different VGGNET Architectures Explored

Table 1: **ConvNet configurations** (shown in columns). The depth of the configurations increases from the left (A) to the right (E), as more layers are added (the added layers are shown in bold). The convolutional layer parameters are denoted as "conv⟨receptive field size⟩-⟨number of channels⟩". The ReLU activation function is not shown for brevity.

ConvNet Configuration					
A	A-LRN	B	C	D	E
11 weight layers	11 weight layers	13 weight layers	16 weight layers	16 weight layers	19 weight layers
input (224×224 RGB image)					
conv3-64	conv3-64 **LRN**	conv3-64 **conv3-64**	conv3-64 conv3-64	conv3-64 conv3-64	conv3-64 conv3-64
maxpool					
conv3-128	conv3-128	conv3-128 **conv3-128**	conv3-128 conv3-128	conv3-128 conv3-128	conv3-128 conv3-128
maxpool					
conv3-256 conv3-256	conv3-256 conv3-256	conv3-256 conv3-256	conv3-256 conv3-256 **conv1-256**	conv3-256 conv3-256 **conv3-256**	conv3-256 conv3-256 conv3-256 **conv3-256**
maxpool					
conv3-512 conv3-512	conv3-512 conv3-512	conv3-512 conv3-512	conv3-512 conv3-512 **conv1-512**	conv3-512 conv3-512 **conv3-512**	conv3-512 conv3-512 conv3-512 **conv3-512**
maxpool					
conv3-512 conv3-512	conv3-512 conv3-512	conv3-512 conv3-512	conv3-512 conv3-512 **conv1-512**	conv3-512 conv3-512 **conv3-512**	conv3-512 conv3-512 conv3-512 **conv3-512**
maxpool					
FC-4096					
FC-4096					
FC-1000					
soft-max					

Parameters in different VGGNET Architectures

Table 2: **Number of parameters (in millions).**

Network	A,A-LRN	B	C	D	E
Number of parameters	133	133	134	138	144

- In spite of a large depth, number of weights is not greater than the number of weights in a more shallow net with larger conv. layer widths and receptive fields

Karen Simonyan, et al, "Very Deep CNNs for Large Scale Image Recognition", ICLR 2015

Performance of VGGNET Architectures

Table 3: **ConvNet performance at a single test scale.**

ConvNet config. (Table 1)	smallest image side		top-1 val. error (%)	top-5 val. error (%)
	train (S)	test (Q)		
A	256	256	29.6	10.4
A-LRN	256	256	29.7	10.5
B	256	256	28.7	9.9
C	256	256	28.1	9.4
	384	384	28.1	9.3
	[256;512]	384	27.3	8.8
D	256	256	27.0	8.8
	384	384	26.8	8.7
	[256;512]	384	25.6	8.1
E	256	256	27.3	9.0
	384	384	26.9	8.7
	[256;512]	384	**25.5**	**8.0**

- Local response normalisation (A-LRN) does not improve on A.
- Classification error decreases with increased depth: 11 (A) to 19 (E).
- Same depth, C (three 1×1 conv. layers), performs worse than configuration D (3×3 conv)
- Additional non-linearity does help (C better than B), however important to capture spatial context by using conv. filters with non-trivial receptive fields (D is better than C).

Training Rescaling & Performance of VGGNET

- Multi-scale training, where each training image is individually rescaled by randomly sampling S from a certain range $[Smin = 256, Smax = 512]$.
- Since objects in images can be of different size, it is beneficial to take this into account during training.
- Training set augmentation by scale jittering, where a single model is trained to recognise objects over a wide range of scales.
- Multi-scale training helps in improving performance.

Karen Simonyan, et al, "Very Deep CNNs for Large Scale Image Recognition", ICLR 2015

Testing Rescaling & Performance of VGGNET

Table 4: **ConvNet performance at multiple test scales.**

ConvNet config. (Table 1)	smallest image side		top-1 val. error (%)	top-5 val. error (%)
	train (S)	test (Q)		
B	256	224,256,288	28.2	9.6
C	256	224,256,288	27.7	9.2
	384	352,384,416	27.8	9.2
	[256; 512]	256,384,512	26.3	8.2
D	256	224,256,288	26.6	8.6
	384	352,384,416	26.5	8.6
	[256; 512]	256,384,512	**24.8**	**7.5**
E	256	224,256,288	26.9	8.7
	384	352,384,416	26.7	8.6
	[256; 512]	256,384,512	**24.8**	**7.5**

- Running a model over several rescaled versions of a test image, followed by averaging resulting class posteriors.
- Scale jittering at training time allows the network to be applied to a wider range of scales at test time.
- Scale jittering at test time leads to better performance.
- Deepest configurations (D and E) perform the best, and scale jittering is better than training with a fixed smallest side S.

ConvNet Fusion & Performance of VGGNET

Table 6: **Multiple ConvNet fusion results.**

Combined ConvNet models	Error		
	top-1 val	top-5 val	top-5 test
ILSVRC submission			
(D/256/224,256,288), (D/384/352,384,416), (D/[256;512]/256,384,512) (C/256/224,256,288), (C/384/352,384,416) (E/256/224,256,288), (E/384/352,384,416)	24.7	7.5	7.3
post-submission			
(D/[256;512]/256,384,512), (E/[256;512]/256,384,512), dense eval.	24.0	7.1	7.0
(D/[256;512]/256,384,512), (E/[256;512]/256,384,512), multi-crop	23.9	7.2	-
(D/[256;512]/256,384,512), (E/[256;512]/256,384,512), multi-crop & dense eval.	**23.7**	**6.8**	**6.8**

- Combine the outputs of several models by averaging their soft-max class posteriors.
- This improves the performance due to complementarity of the models.

Karen Simonyan, et al, "Very Deep CNNs for Large Scale Image Recognition", ICLR 2015

VGGNET vs Others

Table 7: **Comparison with the state of the art in ILSVRC classification**. Our method is denoted as "VGG". Only the results obtained without outside training data are reported.

Method	top-1 val. error (%)	top-5 val. error (%)	top-5 test error (%)
VGG (2 nets, multi-crop & dense eval.)	**23.7**	**6.8**	**6.8**
VGG (1 net, multi-crop & dense eval.)	24.4	7.1	7.0
VGG (ILSVRC submission, 7 nets, dense eval.)	24.7	7.5	7.3
GoogLeNet (Szegedy et al., 2014) (1 net)	-	7.9	
GoogLeNet (Szegedy et al., 2014) (7 nets)	-	**6.7**	
MSRA (He et al., 2014) (11 nets)	-	-	8.1
MSRA (He et al., 2014) (1 net)	27.9	9.1	9.1
Clarifai (Russakovsky et al., 2014) (multiple nets)	-	-	11.7
Clarifai (Russakovsky et al., 2014) (1 net)	-	-	12.5
Zeiler & Fergus (Zeiler & Fergus, 2013) (6 nets)	36.0	14.7	14.8
Zeiler & Fergus (Zeiler & Fergus, 2013) (1 net)	37.5	16.0	16.1
OverFeat (Sermanet et al., 2014) (7 nets)	34.0	13.2	13.6
OverFeat (Sermanet et al., 2014) (1 net)	35.7	14.2	-
Krizhevsky et al. (Krizhevsky et al., 2012) (5 nets)	38.1	16.4	16.4
Krizhevsky et al. (Krizhevsky et al., 2012) (1 net)	40.7	18.2	-

Karen Simonyan, et al, "Very Deep CNNs for Large Scale Image Recognition", ICLR 2015

ResNet : Deep Residual Learning for Image Recognition

- Deeper neural networks are more difficult to train.
- A residual learning framework to ease the training of networks that are substantially deeper than VGGNet
- Explicitly reformulate layers as learning residual functions with reference to layer inputs, instead of learning unreferenced functions.
- Residual networks are easier to optimize, and can gain accuracy from considerably increased depth.

Kaiming He et. al, "Deep Residual Learning for Image Recognition" Proc. ICVPR, 2016

ResNet : Convergence Issue of Deep Networks

- Is learning better networks as easy as stacking more layers?
- Problem of vanishing/exploding gradients, which hamper convergence.
- Largely addressed by normalized initialization and intermediate normalization layers, which enable networks with tens of layers to start converging for stochastic gradient descent (SGD) with backpropagation.

Kaiming He et. al, "Deep Residual Learning for Image Recognition" Proc. ICVPR, 2016

ResNet : Degradation Problem

- When deeper networks are able to start converging, a degradation problem has been exposed.
- With network depth increasing, accuracy gets saturated and then degrades rapidly.
- Unexpectedly, such degradation is not caused by overfitting.
- Adding more layers to a suitably deep model leads to higher training error.

Kaiming He et. al, "Deep Residual Learning for Image Recognition" Proc. ICVPR, 2016

ResNet : MOtivation - Training Error

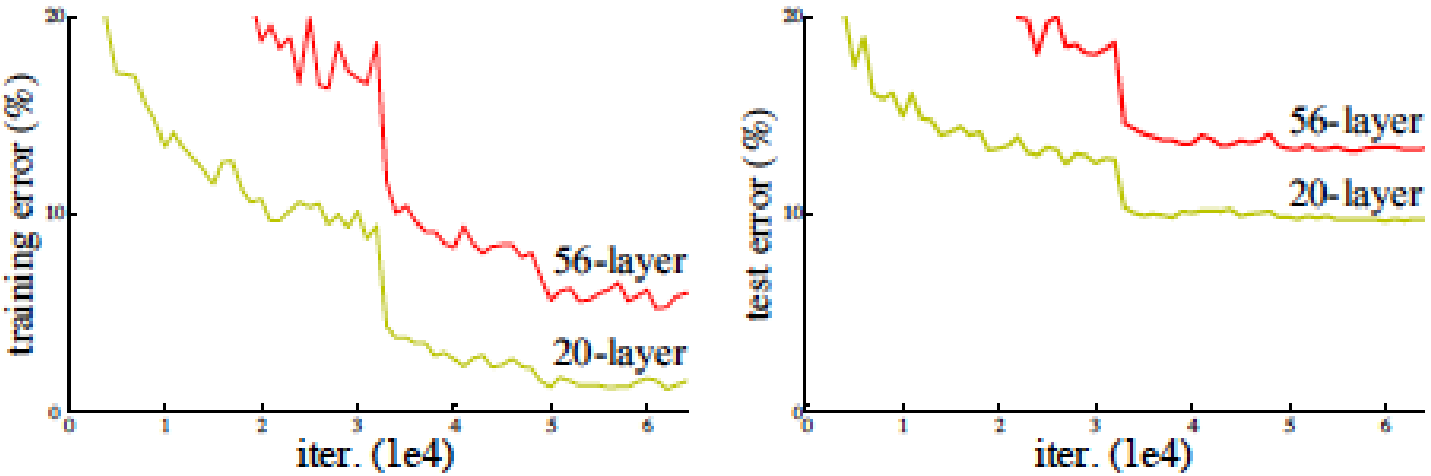


Figure 1. Training error (left) and test error (right) on CIFAR-10 with 20-layer and 56-layer "plain" networks. The deeper network has higher training error, and thus test error. Similar phenomena on ImageNet is presented in Fig. 4.

Kaiming He et. al, "Deep Residual Learning for Image Recognition" Proc. ICVPR, 2016

ResNet : Novelty

- Address degradation problem by introducing a deep residual learning framework.
- Instead of hoping each few stacked layers directly fit a desired underlying mapping, explicitly let these layers fit a residual mapping.
- Denoting desired underlying mapping as $H(x)$, let stacked nonlinear layers fit another mapping of $F(x) := H(x) - x$.
- The original mapping is recast into $F(x) + x$.
- Hypothesize that it is easier to optimize the residual mapping than to optimize the original, unreferenced mapping.

Kaiming He et. al, "Deep Residual Learning for Image Recognition" Proc. ICVPR, 2016

ResNet : Building Block

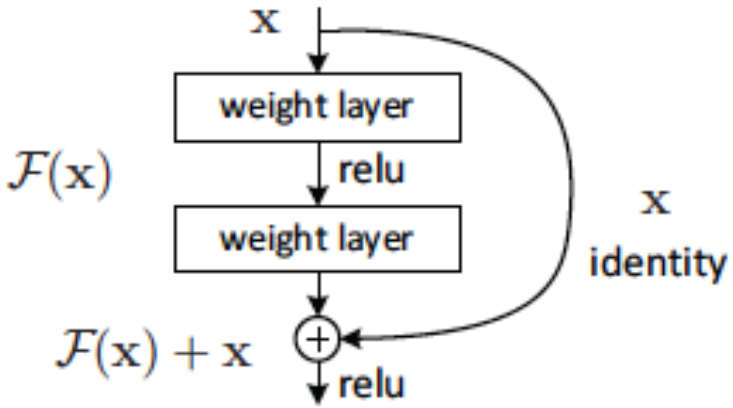


Figure 2. Residual learning: a building block.

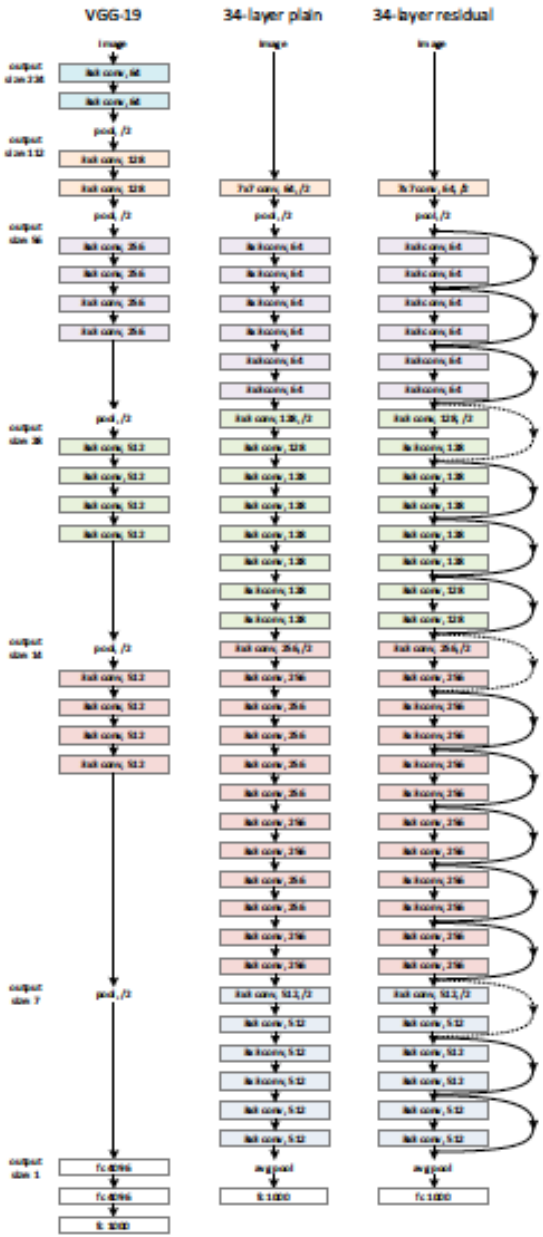
- The formulation of $F(x) + x$ can be realized by feedforward neural networks with "shortcut connections".
- Shortcut connections are those skipping one or more layers.
- In ResNet case, shortcut connections simply perform identity mapping, and their outputs are added to outputs of stacked layers.
- Identity shortcut connections add neither extra parameter nor computational complexity.

ResNet : Proposal

- Extremely deep residual nets are easy to optimize, but the counterpart plain nets (that simply stack layers) exhibit higher training error when the depth increases.
- Deep residual nets can easily enjoy accuracy gains from greatly increased depth, producing results substantially better than previous networks.

Kaiming He et. al, "Deep Residual Learning for Image Recognition" Proc. ICVPR, 2016

ResNet : Network Architecture



ResNet : Training Error

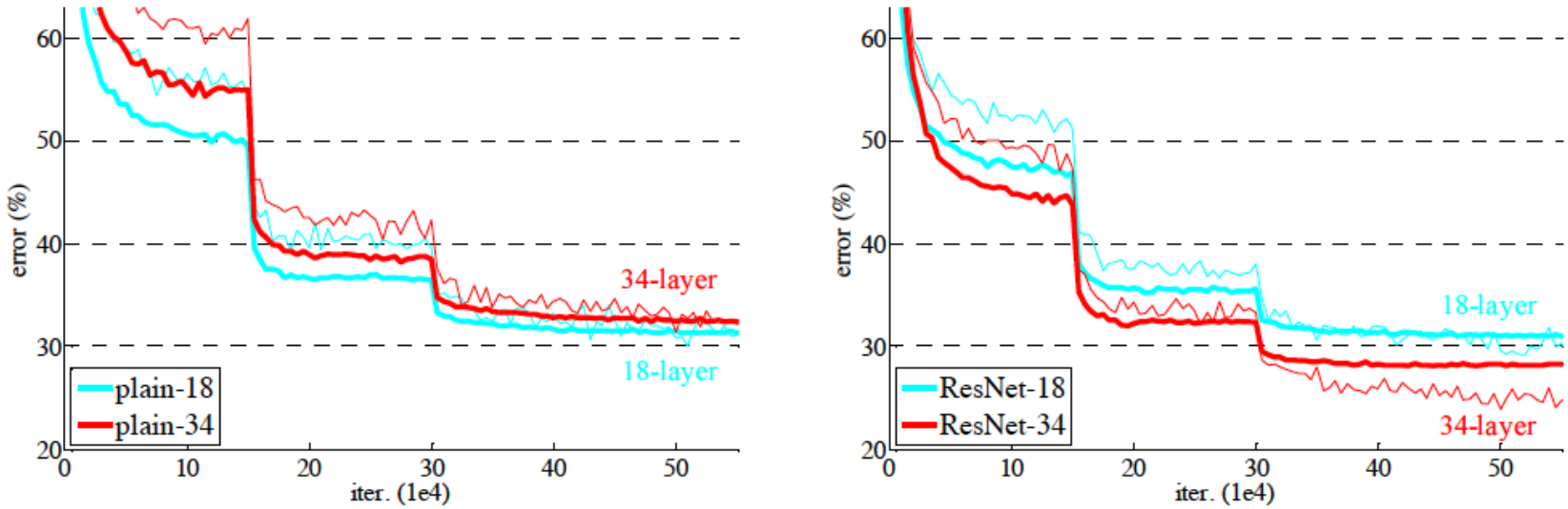


Figure 4. Training on **ImageNet**. Thin curves denote training error, and bold curves denote validation error of the center crops. Left: plain networks of 18 and 34 layers. Right: ResNets of 18 and 34 layers. In this plot, the residual networks have no extra parameter compared to their plain counterparts.

	plain	ResNet
18 layers	27.94	27.88
34 layers	28.54	**25.03**

Table 2. Top-1 error (%, 10-crop testing) on ImageNet validation. Here the ResNets have no extra parameter compared to their plain counterparts. Fig. 4 shows the training procedures.

Kaiming He et. al, "Deep Residual Learning for Image Recognition" Proc. ICVPR, 2016

ResNet : Performance Comparison

model	top-1 err.	top-5 err.
VGG-16 [41]	28.07	9.33
GoogLeNet [44]	-	9.15
PReLU-net [13]	24.27	7.38
plain-34	28.54	10.02
ResNet-34 A	25.03	7.76
ResNet-34 B	24.52	7.46
ResNet-34 C	24.19	7.40
ResNet-50	22.85	6.71
ResNet-101	21.75	6.05
ResNet-152	**21.43**	**5.71**

Table 3. Error rates (%, **10-crop** testing) on ImageNet validation. VGG-16 is based on our test. ResNet-50/101/152 are of option B that only uses projections for increasing dimensions.

Kaiming He et. al, "Deep Residual Learning for Image Recognition" Proc. ICVPR, 2016

ResNet : Ensemble Performance Comparison

method	top-5 err. (**test**)
VGG [41] (ILSVRC'14)	7.32
GoogLeNet [44] (ILSVRC'14)	6.66
VGG [41] (v5)	6.8
PReLU-net [13]	4.94
BN-inception [16]	4.82
ResNet (ILSVRC'15)	**3.57**

Table 5. Error rates (%) of **ensembles**. The top-5 error is on the test set of ImageNet and reported by the test server.

Kaiming He et. al, "Deep Residual Learning for Image Recognition" Proc. ICVPR, 2016

Summary: Journey of CNN in Large Scale Task

Table: Summary of Performance (in %) Different Models in Large Scale Image Recognition Task. AbsI refers absolute improvement. RI1 is relative improvement of Top-5 wrt to previous best system. RI2 is relative improvement wrt to baseline system based on ML (26.2)

Sl. No.	Method	Top-1	Top-5	AbsI	RI1(top5)	RI2(top5)
1	SIFT + FVs		26.2			
2	Alexnet	36.7	15.3	10.9	41	41
3	VGGNet	23.7	6.8	8.5	55	74
4	ResNet		3.57	3.23	49	86

- Significant (step change) improvement in performance due to CNNs
- Significance of learning capacity of CNNs
- Significance of deep models in large scale image recognition task
- Potential of DL models in large scale tasks where large datasets are available

Thank You

[1] I. Goodfellow, Y. Bengio, and A. Courville, *Deep Learning*. MIT Press, 2016.
http://www.deeplearningbook.org.

[2] C. C. Aggarwal *et al.*, "Neural networks and deep learning," *Springer*, vol. 10, no. 978, p. 3, 2018.